

Import necessary packages

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load the file

```
In [2]: income_df=pd.read_csv(r'D:\fsds\projects\datasets\descriptive analysis\Inc_Exp_D
income_df.head()
```

```
Out[2]:
```

	Mthly_HH_Income	Mthly_HH_Expense	No_of_Fly_Members	Emi_or_Rent_Amt	Annua
--	-----------------	------------------	-------------------	-----------------	-------

0	5000	8000	3	2000	
1	6000	7000	2	3000	
2	10000	4500	2	0	
3	10000	2000	1	0	
4	12500	12000	2	3000	



Analyze the Data

```
In [3]: income_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Mthly_HH_Income                       50 non-null    int64
1   Mthly_HH_Expense                       50 non-null    int64
2   No_of_Fly_Members                     50 non-null    int64
3   Emi_or_Rent_Amt                       50 non-null    int64
4   Annual_HH_Income                       50 non-null    int64
5   Highest_Qualified_Member               50 non-null    object
6   No_of_Earning_Members                  50 non-null    int64
dtypes: int64(6), object(1)
memory usage: 2.9+ KB
```

```
In [5]: income_df.shape
```

```
Out[5]: (50, 7)
```

```
In [7]: income_df.describe().T
```

Out[7]:

	count	mean	std	min	25%	50%
Mthly_HH_Income	50.0	41558.00	26097.908979	5000.0	23550.0	35000.0
Mthly_HH_Expense	50.0	18818.00	12090.216824	2000.0	10000.0	15500.0
No_of_Fly_Members	50.0	4.06	1.517382	1.0	3.0	4.0
Emi_or_Rent_Amt	50.0	3060.00	6241.434948	0.0	0.0	0.0
Annual_HH_Income	50.0	490019.04	320135.792123	64200.0	258750.0	447420.0
No_of_Earning_Members	50.0	1.46	0.734291	1.0	1.0	1.0

In [8]: `income_df.isna().any()`

Out[8]:

Mthly_HH_Income	False
Mthly_HH_Expense	False
No_of_Fly_Members	False
Emi_or_Rent_Amt	False
Annual_HH_Income	False
Highest_Qualified_Member	False
No_of_Earning_Members	False

dtype: bool

Mean for expenses of a household

In [9]: `income_df['Mthly_HH_Expense'].mean()`

Out[9]: 18818.0

Median for expense of a household

In [10]: `income_df['Mthly_HH_Expense'].median()`

Out[10]: 15500.0

Mode for expense of a household

In [11]: `income_df['Mthly_HH_Expense'].mode()`

Out[11]: 0 25000
Name: Mthly_HH_Expense, dtype: int64

Monthly expenses for most of the households

```
In [12]: mth_exp_temp=pd.crosstab(index=income_df['Mthly_HH_Expense'], columns='count')
mth_exp_temp.reset_index(inplace=True)
mth_exp_temp[mth_exp_temp['count']==income_df.Mthly_HH_Expense.value_counts().ma
```

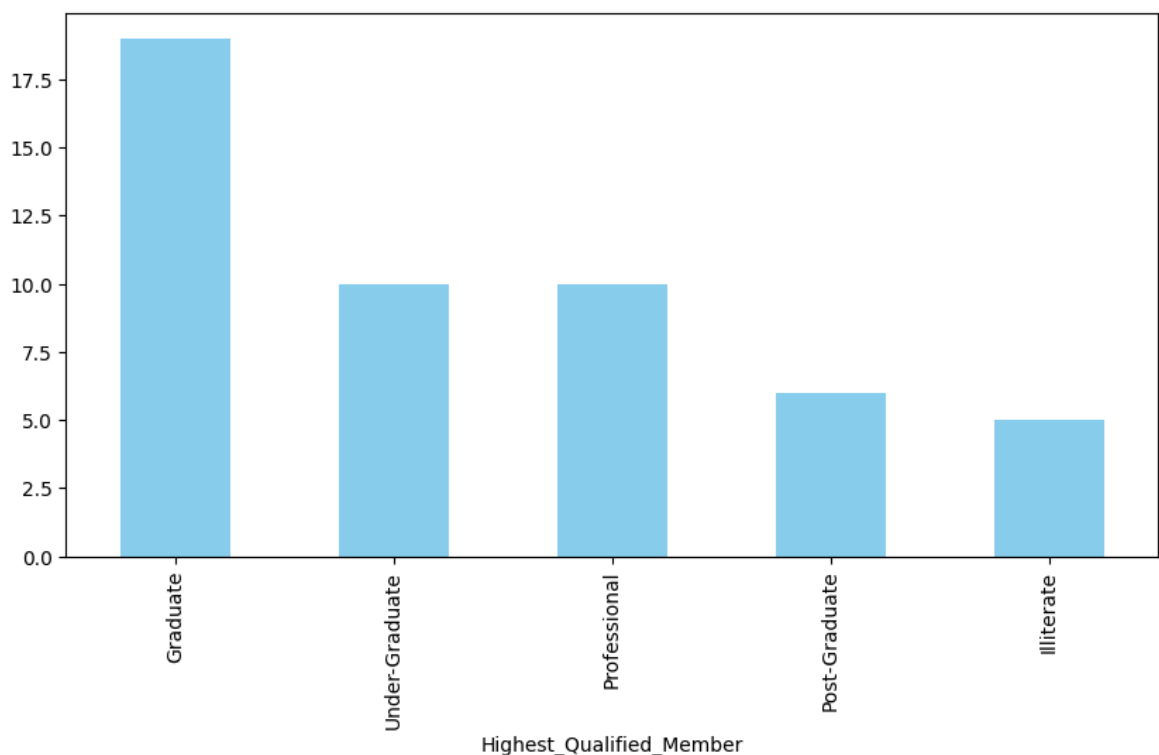
```
Out[12]:
```

col_0	Mthly_HH_Expense	count
18	25000	8

Histogram to count the highest qualified member in a family

```
In [14]: income_df['Highest_Qualified_Member'].value_counts().plot(kind='bar', figsize=(1
```

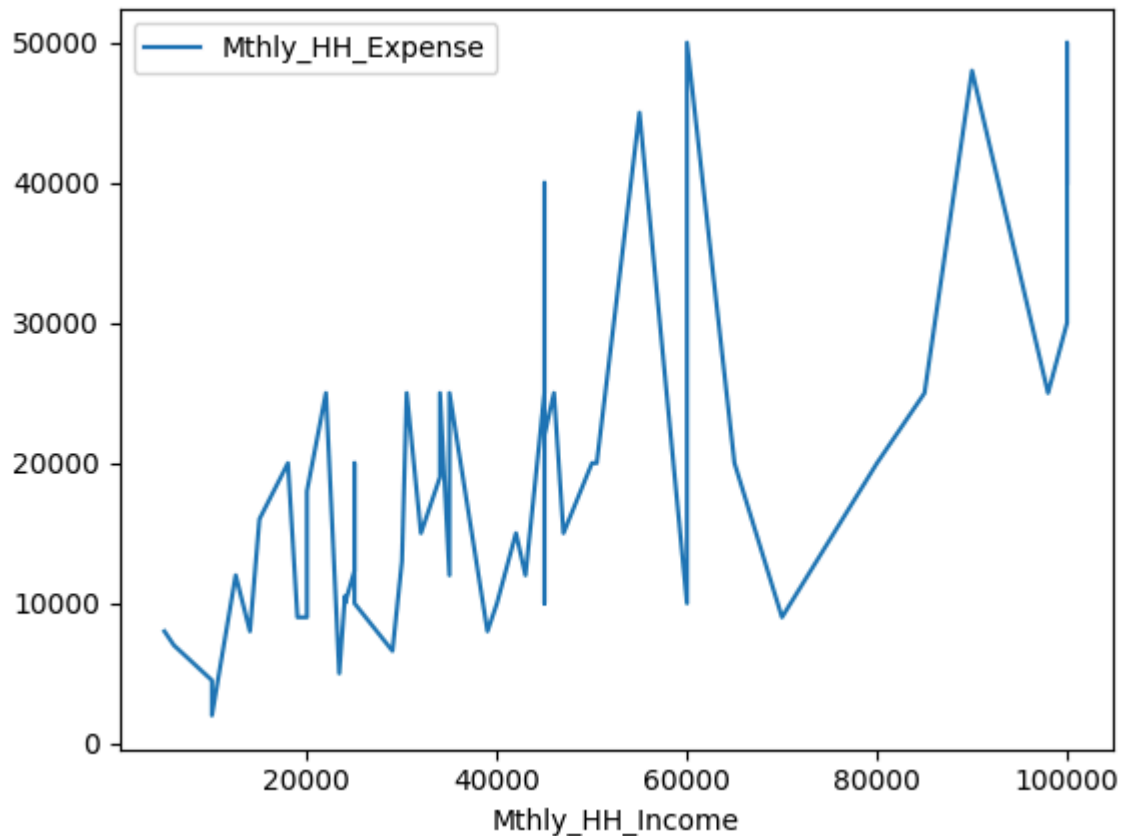
```
Out[14]: <Axes: xlabel='Highest_Qualified_Member'>
```



Calculate IQR(IQR is th difference between 75% and 25% quartile)

```
In [15]: income_df.plot(x='Mthly_HH_Income',y='Mthly_HH_Expense')
IQR= income_df['Mthly_HH_Expense'].quantile(0.75) - income_df['Mthly_HH_Expense']
IQR
```

```
Out[15]: 15000.0
```



Calculating Standard Deviation for fist 4 columns

```
In [17]: pd.DataFrame(income_df.iloc[:,0:5].std().to_frame()).T
```

```
Out[17]:
```

	Mthly_HH_Income	Mthly_HH_Expense	No_of_Fly_Members	Emi_or_Rent_Amt	Annua
0	26097.908979	12090.216824	1.517382	6241.434948	3

Calculating Variance for the first 3 columns

```
In [18]: pd.DataFrame(income_df.iloc[:,0:4].var().to_frame()).T
```

```
Out[18]:
```

	Mthly_HH_Income	Mthly_HH_Expense	No_of_Fly_Members	Emi_or_Rent_Amt
0	6.811009e+08	1.461733e+08	2.302449	3.895551e+07

Calculating the count of Highest qualified member of a family

```
In [19]: income_df['Highest_Qualified_Member'].value_counts().to_frame().T
```

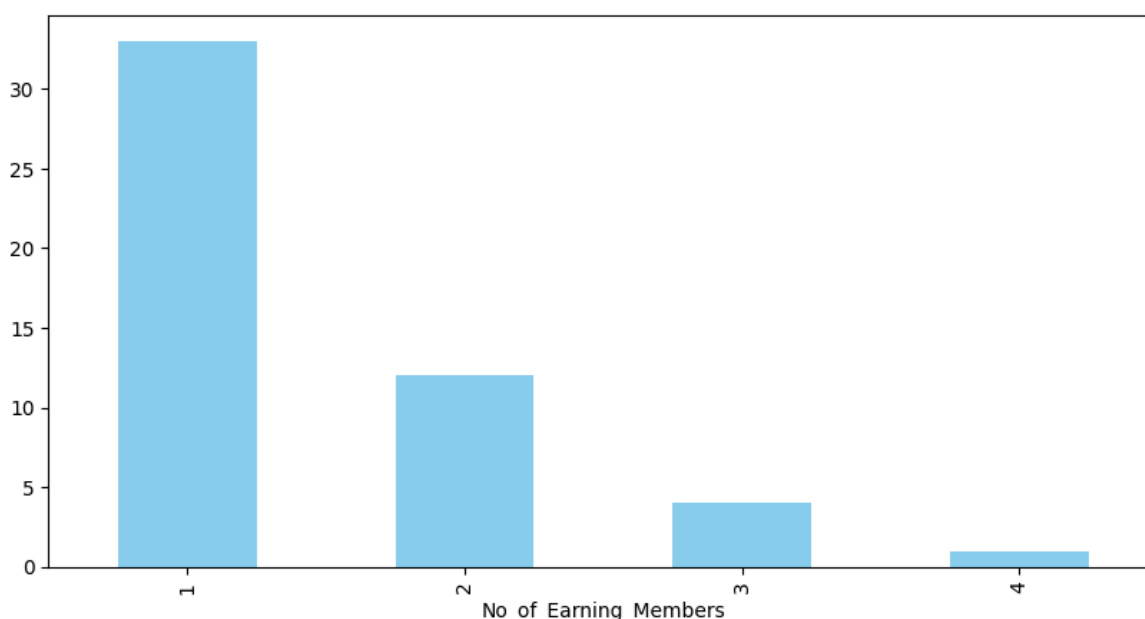
Out[19]:

	Highest_Qualified_Member	Graduate	Under-Graduate	Professional	Post-Graduate	Illiterate
count	19	10	10	6	5	

Plotting the Histogram to count the No_of_Earning_Members

In [21]: `income_df["No_of_Earning_Members"].value_counts().plot(kind="bar", color="skyblue")`

Out[21]: <Axes: xlabel='No_of_Earning_Members'>



Suppose you have option to invest in Stock A or Stock B. The stocks • have different expected returns and standard deviations. The expected return of Stock A is 15% and Stock B is 10%. Standard Deviation of the returns of these stocks is 10% and 5% respectively. Which is the better investment?

In [22]: `Coeff_of_var_StockA=10/15
print(Coeff_of_var_StockA)
Coeff_of_var_StockB=5/10
print(Coeff_of_var_StockB)`

0.6666666666666666
0.5